



CMC UNIVERSITY

Aspire to Inspire the Digital World

CHƯƠNG 6: JavaScript

Khoa CNTT&TT - Trường Đại học CMC

Hà Nội, tháng 10 năm 2025

- Giới thiệu Javascript
- Cú pháp cơ bản
- Biến và kiểu dữ liệu
- Toán tử (Operators)
- Cấu trúc điều khiển
- Vòng lặp
- Hàm (Function)
- Object (Đối tượng)
- Array (Mảng)
- RegularExpression
- DOM (Document Object Model)
- DOM Event
- Lập trình bất đồng bộ (async)
- Modules – ES6 – tính năng hiện đại
- Quản lý dữ liệu trên trình duyệt
- Ứng dụng javascript vào web thực

Your Website

```
graph TD; A[Your Website] --- B[HTML]; A --- C[CSS]; A --- D[JavaScript]; B --- E[What do you want on your website?]; C --- F[How do you want it to look?]; D --- G[What do you want it to do?];
```

HTML

What do you want
on your website?

CSS

How do you want
it to look?

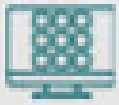
JavaScript

What do you want
it to do?

- **JavaScript** là một ngôn ngữ lập trình được sử dụng chủ yếu để phát triển các ứng dụng web. Nó cho phép các nhà phát triển tạo ra các trang web tương tác, động và tương tác với người dùng một cách linh hoạt hơn. JavaScript là một trong ba công nghệ cốt lõi của web cùng với HTML và CSS.



Website



Desktop App



Mobile App



Chatbots



Game



Robots and IoT



Smartwatch App



Server-Side App

Applications of Javascript

- Lịch sử và phát triển của JavaScript.
- Vai trò của JavaScript trong phát triển web (Frontend/Backend).
- So sánh JavaScript với các ngôn ngữ khác như Python, Java, C++.
- Sự kết hợp giữa HTML, CSS, và JavaScript.

Các khái niệm cơ bản

Cú pháp cơ bản



Biến và kiểu dữ liệu



Cấu trúc điều khiển

- Viết JavaScript trong HTML:
 - Cách nhúng JavaScript vào HTML qua thẻ `<script>`.
- Lệnh cơ bản:
 - `console.log()`
 - `document.write()`
- Comment trong JavaScript:
 - Comment dòng đơn `//`
 - Comment nhiều dòng `/* */`.

Javascript Keywords



KEYWORD	DESCRIPTION
var	Declares a variable
let	Declares a block variable
const	Declares a block constant
if	Marks a block of statements to be executed on a condition
switch	Marks a block of statements to be executed in different cases
for	Marks a block of statements to be executed in a loop
function	Declares a function
return	Exits a function
try	Implements error handling to a block of statements

- Khai báo biến: Sử dụng **var**, **let**, và **const** để khai báo biến.
 - var: Phạm vi toàn cục hoặc trong hàm.
 - let: Phạm vi trong khối.
 - const: Khai báo hằng số.
- Kiểu dữ liệu:
 - Number: Số nguyên, số thực.
 - String: Chuỗi.
 - Boolean: Giá trị đúng/sai.
 - Object: Đối tượng.
 - Array: Mảng.
 - Undefined: Giá trị chưa xác định.

- **JavaScript Operators** (Toán tử trong JavaScript) là các ký hiệu hoặc từ khóa dùng để thực hiện các phép toán trên các giá trị, biến, hoặc biểu thức. Các toán tử có thể được chia thành nhiều loại khác nhau, như toán tử số học, logic, so sánh, gán, và nhiều loại khác.
- Các loại toán tử
 - Toán tử số học
 - Toán tử gán
 - Toán tử so sánh
 - Toán tử logic
 - Toán tử kiểu dữ liệu

Toán tử số học (Arithmetic Operators)

Được sử dụng để thực hiện các phép toán số học trên các số.

Toán tử	Tên	Ví dụ	Kết quả
+	Cộng	<code>5 + 2</code>	7
-	Trừ	<code>5 - 2</code>	3
*	Nhân	<code>5 * 2</code>	10
/	Chia	<code>5 / 2</code>	2.5
%	Chia lấy dư	<code>5 % 2</code>	1
++	Tăng lên 1	<code>let a = 5; a++</code>	6
--	Giảm đi 1	<code>let a = 5; a--</code>	4

Toán tử gán (Assignment Operators)

Dùng để gán giá trị cho một biến.

Toán tử	Tên	Ví dụ	Kết quả
=	Gán	<code>x = 10</code>	<code>x = 10</code>
+=	Cộng và gán	<code>x += 5</code>	<code>x = x + 5</code>
-=	Trừ và gán	<code>x -= 5</code>	<code>x = x - 5</code>
*=	Nhân và gán	<code>x *= 5</code>	<code>x = x * 5</code>
/=	Chia và gán	<code>x /= 5</code>	<code>x = x / 5</code>
%=	Chia lấy dư và gán	<code>x %= 5</code>	<code>x = x % 5</code>

Toán tử so sánh (Comparison Operators)

Dùng để so sánh hai giá trị, trả về giá trị **true** hoặc **false**.

Toán tử	Tên	Ví dụ	Kết quả
<code>==</code>	So sánh bằng	<code>5 == 5</code>	<code>true</code>
<code>===</code>	So sánh bằng tuyệt đối (cả kiểu dữ liệu)	<code>5 === '5'</code>	<code>false</code>
<code>!=</code>	Khác nhau	<code>5 != 3</code>	<code>true</code>
<code>!==</code>	Khác nhau tuyệt đối	<code>5 !== '5'</code>	<code>true</code>
<code>></code>	Lớn hơn	<code>5 > 3</code>	<code>true</code>
<code><</code>	Nhỏ hơn	<code>3 < 5</code>	<code>true</code>
<code>>=</code>	Lớn hơn hoặc bằng	<code>5 >= 5</code>	<code>true</code>
<code><=</code>	Nhỏ hơn hoặc bằng	<code>3 <= 5</code>	<code>true</code>

Toán tử logic (Logical Operators)

Được sử dụng để kết hợp các biểu thức logic, trả về giá trị **true** hoặc **false**.

Toán tử	Tên	Ví dụ	Kết quả
&&	Và (AND)	true && false	false
,		,	Hoặc (OR)
!	Phủ định (NOT)	!true	false

Toán tử chuỗi (String Operators)

Sử dụng để nối các chuỗi.

Toán tử	Tên	Ví dụ	Kết quả
+	Nối chuỗi	'Hello' + ' World'	'Hello World'

Toán tử điều kiện (Conditional/Ternary Operator)

Toán tử ba ngôi (`? :`) được sử dụng để thay thế cho câu lệnh điều kiện `if...else` .

Toán tử	Tên	Ví dụ	Kết quả
<code>? :</code>	Toán tử ba ngôi	<code>age >= 18 ? "Adult" : "Minor"</code>	<code>"Adult"</code> nếu <code>age >= 18</code> , ngược lại <code>"Minor"</code>

Toán tử bitwise (Bitwise Operators)

Được sử dụng để thao tác trực tiếp trên các bit nhị phân của số nguyên.

Toán tử	Tên	Ví dụ	Kết quả
&	AND bit	5 & 1	1
	OR bit	5 1	5
^	XOR bit	5 ^ 1	4
~	NOT bit	~5	-6
<<	Dịch trái	5 << 1	10
>>	Dịch phải	5 >> 1	2

Toán tử kiểu dữ liệu (Type Operators)

Toán tử	Tên	Ví dụ	Kết quả
<code>typeof</code>	Kiểm tra kiểu dữ liệu	<code>typeof 5</code>	<code>'number'</code>
<code>instanceof</code>	Kiểm tra đối tượng có thuộc lớp nào không	<code>obj instanceof Array</code>	<code>true</code> hoặc <code>false</code>

Cấu trúc điều khiển

Câu lệnh điều kiện

- if,
- else if,
- else

Vòng lặp

- for,
- while,
- do...while

- **Câu lệnh điều kiện trong JavaScript** cho phép bạn kiểm soát luồng thực thi của chương trình dựa trên kết quả của các biểu thức điều kiện. Các câu lệnh này thường được sử dụng để thực hiện các thao tác khác nhau tùy thuộc vào điều kiện đúng hoặc sai.
 - **if**: Kiểm tra điều kiện và thực thi khối lệnh nếu điều kiện đúng.
 - **if...else**: Thực thi khối lệnh khác nếu điều kiện sai.
 - **if...else if...else**: Kiểm tra nhiều điều kiện.
 - **Toán tử ba ngôi (?:)**: Viết tắt của if...else cho các điều kiện đơn giản.
 - **switch**: Dùng để kiểm tra nhiều giá trị của một biểu thức.

- **Vòng lặp trong JavaScript** được sử dụng để thực thi một khối lệnh nhiều lần dựa trên điều kiện cụ thể. Vòng lặp giúp giảm thiểu việc viết lặp lại các dòng lệnh giống nhau, đồng thời làm cho mã nguồn trở nên ngắn gọn và hiệu quả hơn.
- Các kiểu lặp
 - **Vòng lặp for**: Sử dụng khi biết trước số lần lặp.
 - **Vòng lặp while**: Sử dụng khi số lần lặp không biết trước, và lặp lại khi điều kiện đúng.
 - **Vòng lặp do...while**: Tương tự while, nhưng luôn thực thi ít nhất một lần.
 - **Vòng lặp for...of**: Lặp qua các giá trị của mảng, chuỗi, hoặc đối tượng có thể lặp lại.
 - **Vòng lặp for...in**: Lặp qua các thuộc tính của đối tượng.
- Câu lệnh **break** và **continue**
 - Được sử dụng để điều khiển luồng vòng lặp

Hàm và đối tượng

Hàm (Functions)

- **Hàm cơ bản:** Cách khai báo hàm, tham số, giá trị trả về
- Hàm mũi tên

Đối tượng (Objects)

- **Tạo đối tượng:** Cách tạo đối tượng với thuộc tính và phương thức

- **JavaScript Functions** (Hàm trong JavaScript) là các khối mã có thể được gọi lại để thực thi các tác vụ. Hàm giúp bạn tái sử dụng mã và giảm thiểu sự lặp lại trong chương trình. Chúng có thể nhận tham số và trả về kết quả, hoặc chỉ thực hiện các hành động mà không cần trả về gì.
- Cách đơn giản nhất để khai báo một hàm trong JavaScript là sử dụng từ khóa **function**.
- Hàm có thể trả về một giá trị bằng từ khóa **return**. Khi gặp lệnh return, hàm sẽ dừng lại và trả về giá trị đó.

- Hàm có thể nhận các **tham số** (parameters), đó là các giá trị mà bạn truyền vào khi gọi hàm.

- Ví dụ

//a, b là tham số

```
function add(a, b) {  
    return a + b;  
}
```

```
let result = add(5, 3); // Gọi hàm với đối số 5 và 3  
console.log(result); // Kết quả: 8
```

- Bạn cũng có thể gán một hàm cho một biến. Đây được gọi là **biểu thức hàm** (function expression).

```
javascript Sao chép mã  
  
let greet = function(name) {  
    return "Hello, " + name;  
};  
  
console.log(greet("John")); // Kết quả: Hello, John
```

- Lưu ý: Trong **biểu thức hàm**, hàm không bắt buộc phải có tên, vì vậy hàm này còn được gọi là **hàm ẩn danh** (anonymous function).

- Hàm gọi lại là hàm được truyền vào như một đối số cho hàm khác và được gọi lại bên trong hàm đó. Hàm gọi lại rất phổ biến trong JavaScript, đặc biệt là trong xử lý bất đồng bộ (asynchronous).

javascript

Sao chép mã

```
function doSomething(callback) {  
    console.log("Doing something...");  
    callback();  
}  
  
function sayGoodbye() {  
    console.log("Goodbye!");  
}  
  
doSomething(sayGoodbye);
```

- **JavaScript Objects** là một phần quan trọng của lập trình JavaScript, được sử dụng để lưu trữ các tập hợp dữ liệu và các giá trị có liên quan. Đối tượng (object) trong JavaScript là một kiểu dữ liệu phức tạp, cho phép bạn nhóm các giá trị khác nhau dưới dạng cặp **key-value**. Các đối tượng có thể chứa dữ liệu (thuộc tính) và chức năng (phương thức) để mô tả các thực thể hoặc khái niệm trong đời sống thực.
- Có nhiều cách để tạo đối tượng trong JavaScript. Cách phổ biến nhất là sử dụng cặp dấu ngoặc nhọn {} để tạo một đối tượng.

```
let person = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};
```

- Bạn có thể truy cập các thuộc tính của một đối tượng bằng hai cách:
 - Dấu chấm (.).
 - Dấu ngoặc vuông ([]).

```
// Truy cập bằng dấu chấm
console.log(person.name); // Kết quả: John

// Truy cập bằng dấu ngoặc vuông
console.log(person["age"]); // Kết quả: 30
```

- Bạn có thể thay đổi giá trị của một thuộc tính, hoặc thêm một thuộc tính mới vào đối tượng.

```
// Thay đổi giá trị thuộc tính
person.age = 35;
console.log(person.age); // Kết quả: 35

// Thêm thuộc tính mới
person.country = "USA";
console.log(person.country); // Kết quả: USA
```

- Xóa thuộc tính

```
delete person.city;
console.log(person.city); // Kết quả: undefined
```

- Phương thức là một hàm được định nghĩa bên trong đối tượng. Nó có thể truy cập các thuộc tính của đối tượng bằng từ khóa `this`.

```
let person = {  
  name: "John",  
  age: 30,  
  greet: function() {  
    return "Hello, my name is " + this.name;  
  }  
};  
  
console.log(person.greet()); // Kết quả: Hello, my name is John
```

- Ngoài cách khai báo đối tượng trực tiếp, bạn cũng có thể tạo đối tượng bằng cách sử dụng hàm tạo (constructor). Đây là một cách tiện lợi để tạo nhiều đối tượng có cùng cấu trúc.

```
function Person(name, age, city) {  
  this.name = name;  
  this.age = age;  
  this.city = city;  
}  
  
let person1 = new Person("John", 30, "New York");  
let person2 = new Person("Jane", 25, "Los Angeles");  
  
console.log(person1.name); // Kết quả: John  
console.log(person2.name); // Kết quả: Jane
```


- JavaScript cung cấp một số phương thức có sẵn để làm việc với đối tượng, như:
 - `Object.keys()`: Trả về một mảng chứa tất cả các tên thuộc tính của đối tượng.
 - `Object.values()`: Trả về một mảng chứa tất cả các giá trị thuộc tính của đối tượng.
 - `Object.entries()`: Trả về một mảng chứa các cặp `[key, value]`.

```
let person = {  
  name: "John",  
  age: 30,  
  city: "New York"  
};  
  
console.log(Object.keys(person)); // Kết quả: ["name", "age", "city"]  
console.log(Object.values(person)); // Kết quả: ["John", 30, "New York"]  
console.log(Object.entries(person)); // Kết quả: [["name", "John"], ["age", 30], ["city",
```

- Bạn có thể sử dụng vòng lặp for...in để lặp qua tất cả các thuộc tính của một đối tượng.

javascript

Sao chép mã

```
for (let key in person) {  
  console.log(key + ": " + person[key]);  
}
```

Kết quả:

vbnet

Sao chép mã

```
name: John  
age: 30  
city: New York
```



1. Tạo một Object tên là student có các thuộc tính sau:
 - name: Tên của sinh viên (chuỗi).
 - age: Tuổi của sinh viên (số).
 - major: Ngành học của sinh viên (chuỗi).
2. Truy cập và in ra console giá trị của thuộc tính name và major.
3. Thêm thuộc tính gpa (điểm trung bình) cho Object student đã tạo ở trên và gán giá trị cho thuộc tính này.
4. Chỉnh sửa giá trị thuộc tính age thành 22.
5. In lại toàn bộ Object sau khi chỉnh sửa.
6. Thêm một phương thức vào Object student tên là introduce để in ra lời giới thiệu như sau: "My name is [name], I am [age] years old and I study [major]".
7. Gọi phương thức introduce và kiểm tra kết quả.
8. Xóa thuộc tính major khỏi Object student. In lại Object sau khi xóa để kiểm tra.
9. Tạo một Object university có các thuộc tính:
 - name: Tên trường đại học.
 - location: Địa chỉ trường đại học.
 - students: Một mảng chứa danh sách các Object sinh viên (tương tự như Object student đã tạo ở bài trước).
10. Truy cập vào tên của trường đại học và in ra danh sách tên của tất cả sinh viên trong mảng students.
11. Sử dụng phương thức Object.assign hoặc toán tử spread (...) để sao chép Object student.
12. Thay đổi giá trị thuộc tính name của Object sao chép mà không ảnh hưởng đến Object gốc.

Bạn được yêu cầu phát triển một hệ thống để xử lý các đoạn văn bản tự động cho một ứng dụng blog. Người dùng có thể viết bài viết nhưng đôi khi bài viết không đúng định dạng và chứa nhiều thông tin không cần thiết. Bạn cần viết một tập hợp các chức năng để xử lý chuỗi trong các tình huống sau:

1. Viết một hàm `shortenText(text, maxLength)` để rút ngắn đoạn văn bản nếu nó dài hơn `maxLength` ký tự. Nếu văn bản bị rút ngắn, hãy thêm ... vào cuối chuỗi.

Ví dụ:

- Input: `shortenText("JavaScript is a versatile language used for both front-end and back-end development", 30);`
 - Output: `"JavaScript is a versatile la..."`
2. Viết hàm `countWordsAndSentences(text)` để đếm số từ và số câu trong một đoạn văn bản. Một câu được xem là kết thúc khi gặp các dấu chấm (.), dấu chấm hỏi (?), hoặc dấu chấm than (!).
 3. Viết hàm `wordFrequency(text)` để thống kê số lần xuất hiện của mỗi từ trong đoạn văn bản. Từ được so sánh không phân biệt chữ hoa và chữ thường.

Ví dụ:

- Input: `wordFrequency("Hello hello world. The world is vast and full of opportunities.");`
- Output: `{hello: 2, world: 2, the: 1...}`

Bạn được yêu cầu phát triển một tập hợp các chức năng để quản lý dữ liệu học viên trong một lớp học. Mỗi học viên có thông tin như tên, tuổi, và điểm số. Bạn sẽ làm việc với danh sách mảng các học viên để thực hiện các thao tác dưới đây.

1. Viết hàm `addStudent(students, student)` để thêm một học viên mới vào danh sách học viên
2. Viết hàm `findStudentByName(students, name)` để tìm học viên theo tên
3. Viết hàm `removeStudentByName(students, name)` để xóa học viên theo tên
4. Viết hàm `calculateAverageScore(students)` để tính điểm trung bình của tất cả học viên trong lớp.
5. Viết hàm `sortStudentsByScore(students)` để sắp xếp danh sách học viên theo điểm số từ cao xuống thấp.
6. Viết hàm `filterStudentsByAge(students, minAge)` để lọc các học viên có tuổi lớn hơn hoặc bằng `minAge`.
7. Viết hàm `hasStudentBelowAverage(students, averageScore)` để kiểm tra xem có học viên nào có điểm thấp hơn điểm trung bình (`averageScore`) không.

Mảng và chuỗi

Mảng (Arrays)

- **Khai báo mảng:** Cách tạo mảng và truy cập phần tử.
- **Phương thức của mảng:** push(), pop(), shift(), unshift(), splice(), slice(), map(), filter(), forEach().

Chuỗi (Strings)

- **Phương thức xử lý chuỗi:** length, slice(), indexOf(), replace(), toUpperCase(), toLowerCase().

- **JavaScript Arrays** (Mảng trong JavaScript) là một kiểu dữ liệu đặc biệt, dùng để lưu trữ danh sách các phần tử (phần tử có thể là số, chuỗi, đối tượng hoặc thậm chí các mảng khác). Mảng trong JavaScript rất linh hoạt và cung cấp nhiều phương thức mạnh mẽ để thao tác dữ liệu.

```
let numbers = [1, 2, 3, 4, 5];  
let fruits = ["Apple", "Banana", "Orange"];
```

- Mỗi phần tử trong mảng được truy cập bằng chỉ số (index), chỉ số bắt đầu từ 0.

```
let fruits = ["Apple", "Banana", "Orange"];  
console.log(fruits[0]); // Kết quả: Apple  
console.log(fruits[2]); // Kết quả: Orange
```


- Bạn có thể thay đổi giá trị của một phần tử trong mảng bằng cách gán giá trị mới cho chỉ số của phần tử đó.

```
let fruits = ["Apple", "Banana", "Orange"];  
fruits[1] = "Mango"; // Thay đổi phần tử thứ 2  
console.log(fruits); // Kết quả: ["Apple", "Mango", "Orange"]
```

- Bạn có thể lấy độ dài của mảng (số lượng phần tử) bằng thuộc tính length.

```
let fruits = ["Apple", "Banana", "Orange"];  
console.log(fruits.length); // Kết quả: 3
```

- `push()`: Thêm phần tử vào cuối mảng

```
let fruits = ["Apple", "Banana"];  
fruits.push("Orange");  
console.log(fruits); // Kết quả: ["Apple", "Banana", "Orange"]
```

- `pop()`: Xóa phần tử cuối cùng khỏi mảng

```
let fruits = ["Apple", "Banana", "Orange"];  
fruits.pop();  
console.log(fruits); // Kết quả: ["Apple", "Banana"]
```

- `unshift()`: Thêm phần tử vào đầu mảng

```
let fruits = ["Banana", "Orange"];  
fruits.unshift("Apple");  
console.log(fruits); // Kết quả: ["Apple", "Banana", "Orange"]
```

- `shift()`: Xóa phần tử đầu tiên khỏi mảng

```
let fruits = ["Apple", "Banana", "Orange"];  
fruits.shift();  
console.log(fruits); // Kết quả: ["Banana", "Orange"]
```

- `splice()`: Xóa hoặc thêm phần tử vào giữa mảng

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.splice(1, 1, "Mango"); // Xóa 1 phần tử tại vị trí 1 và thêm "Mango"
console.log(fruits); // Kết quả: ["Apple", "Mango", "Orange"]
```

- `slice()`: Trích xuất một phần của mảng thành mảng mới

```
let fruits = ["Apple", "Banana", "Orange", "Mango"];
let citrus = fruits.slice(1, 3);
console.log(citrus); // Kết quả: ["Banana", "Orange"]
```

- `concat()`: Nối hai hoặc nhiều mảng lại với nhau

```
let fruits = ["Apple", "Banana"];  
let moreFruits = ["Orange", "Mango"];  
let allFruits = fruits.concat(moreFruits);  
console.log(allFruits); // Kết quả: ["Apple", "Banana", "Orange", "Mango"]
```

- `indexOf()` và `lastIndexOf()`: Tìm vị trí của một phần tử trong mảng

```
let fruits = ["Apple", "Banana", "Orange"];  
console.log(fruits.indexOf("Banana")); // Kết quả: 1
```

- Bạn có thể sử dụng các vòng lặp như for, for...of, hoặc các phương thức như forEach() để duyệt qua các phần tử của mảng.

```
let fruits = ["Apple", "Banana", "Orange"];
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

```
let fruits = ["Apple", "Banana", "Orange"];
for (let fruit of fruits) {
  console.log(fruit);
}
```

```
let fruits = ["Apple", "Banana", "Orange"];
fruits.forEach(function(fruit) {
  console.log(fruit);
});
```

- `map()`: Tạo ra một mảng mới từ việc thực hiện một thao tác trên mỗi phần tử của mảng.

```
let numbers = [1, 2, 3, 4];  
let squares = numbers.map(num => num * num);  
console.log(squares); // Kết quả: [1, 4, 9, 16]
```

- `filter()`: Tạo ra một mảng mới với các phần tử thỏa mãn điều kiện.

```
let numbers = [1, 2, 3, 4, 5];  
let evenNumbers = numbers.filter(num => num % 2 === 0);  
console.log(evenNumbers); // Kết quả: [2, 4]
```

- `reduce()`: Giảm mảng thành một giá trị duy nhất.

```
let numbers = [1, 2, 3, 4];  
let sum = numbers.reduce((total, num) => total + num, 0);  
console.log(sum); // Kết quả: 10
```


- **Ví dụ 1: Thêm, xóa và duyệt qua mảng**

Giả sử bạn có một mảng chứa danh sách các tên và muốn thực hiện các thao tác như thêm tên mới, xóa tên và duyệt qua các phần tử trong mảng.

- **Bài tập:**

1. Tạo một mảng chứa danh sách các tên.
2. Thêm một tên mới vào mảng.
3. Xóa tên cuối cùng khỏi mảng.
4. Duyệt qua mảng và in ra từng tên.

- **Ví dụ 2: Tìm số lớn nhất trong mảng**

Cho một mảng các số, tìm số lớn nhất trong mảng.

- **Bài tập:**

1. Tạo một mảng chứa các số.
2. Sử dụng vòng lặp để tìm số lớn nhất trong mảng.

- Cho một mảng chứa các số
 - Tính tổng của các phần tử trong mảng.
 - Liệt kê các số chẵn trong mảng
 - Liệt kê các số lẻ trong mảng

- Regular Expression (Regex) là **mẫu (pattern)** dùng để:
 - Tìm kiếm chuỗi
 - Kiểm tra (validate) dữ liệu
 - Thay thế nội dung
 - Trích xuất thông tin
- JavaScript hỗ trợ Regex thông qua:
 - Đối tượng RegExp
 - Các hàm String như `.match()`, `.replace()`, `.test()`

Dạng literal

- `const regex = /abc/;`
-

Dạng constructor

- `const regex = new RegExp("abc");`
-

Dùng constructor khi cần tạo regex động.

Ký hiệu	Ý nghĩa
.	Bất kỳ ký tự nào
\d	Chữ số (0–9)
\w	Ký tự chữ + số + _
\s	Khoảng trắng
^	Bắt đầu chuỗi
\$	Kết thúc chuỗi
+	Lặp ≥ 1 lần
*	Lặp ≥ 0 lần
?	0 hoặc 1 lần / không tham lam

Flag	Mô tả
g	global – tìm tất cả
i	ignoreCase – không phân biệt hoa/thường
m	multiline – nhiều dòng
u	unicode
s	cho phép . khớp với ký tự xuống dòng

test() – kiểm tra khớp

- `/abc/.test("abcde"); // true`
-

match() – trả về mảng kết quả

- `"hello123".match(/\d+/); // ["123"]`
-

replace() – thay thế

- `"2025-01-01".replace(/-/g, "/");`
- `// "2025/01/01"`
-

split() – tách chuỗi theo regex

- `"a,b;c".split(/[;,]/); // ["a","b","c"]`


```
const regex = /(\w+)\@(\w+)\.com/;  
const result = "test@gmail.com".match(regex);  
// result[1] = "test"  
// result[2] = "gmail"
```

- Dùng ngoặc () để nhóm
- Dùng ? : để tạo **non-capturing group**: (? : abc)

Kiểm tra email

- `const emailRegex = /^[\\w.-]+@[\\w.-]+\\.\\w+$/;`
-

Tìm số trong chuỗi

- `"abc123".match(/\\d+/g); // ["123"]`
-

Loại bỏ ký tự đặc biệt

- `text.replace(/[^\\w\\s]/g, "");`

- Regex mạnh nhưng **khó đọc**
- Không nên lạm dụng khi có cách xử lý chuỗi đơn giản hơn
- Nên kiểm tra regex bằng:
 - regex101.com
 - regexr.com

DOM (Document Object Model)

DOM là gì?

- **Khái niệm DOM:** DOM là mô hình tài liệu, cho phép JavaScript tương tác với HTML/CSS.
- **Cách truy cập phần tử:** getElementById(), querySelector(), getElementsByClassName().

Thao tác với DOM

- **Thay đổi nội dung HTML:** Sử dụng innerHTML, textContent.
- **Thay đổi thuộc tính CSS:** style, classList.
- **Sự kiện DOM:** click, mouseover, keyup...

Tương tác với sự kiện

- Sự kiện trong JavaScript
 - Các loại sự kiện: click, mouseover, input, change, submit.
 - Thêm sự kiện bằng addEventListener() và xóa sự kiện bằng removeEventListener().

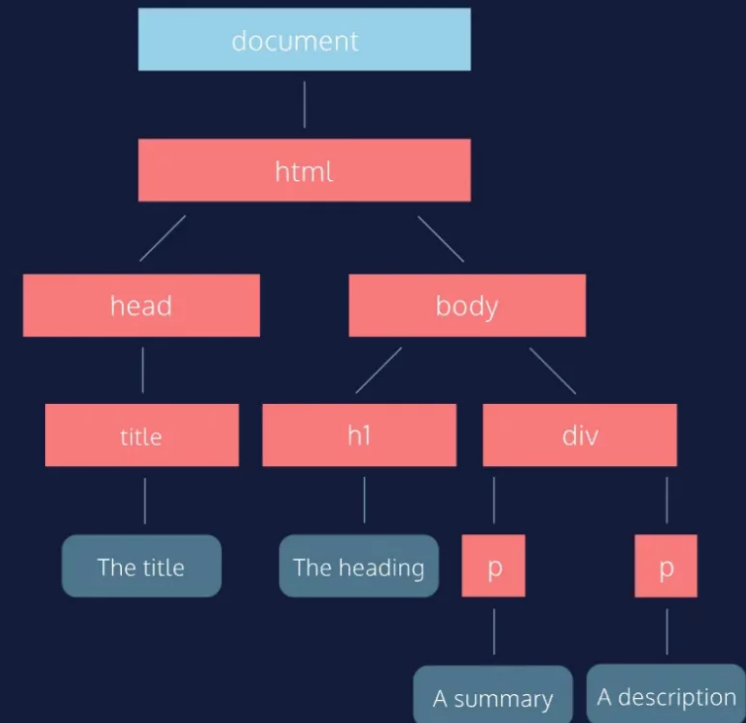
HTML File

```
<html>
  <head>
    <title>The title</title>
  </head>

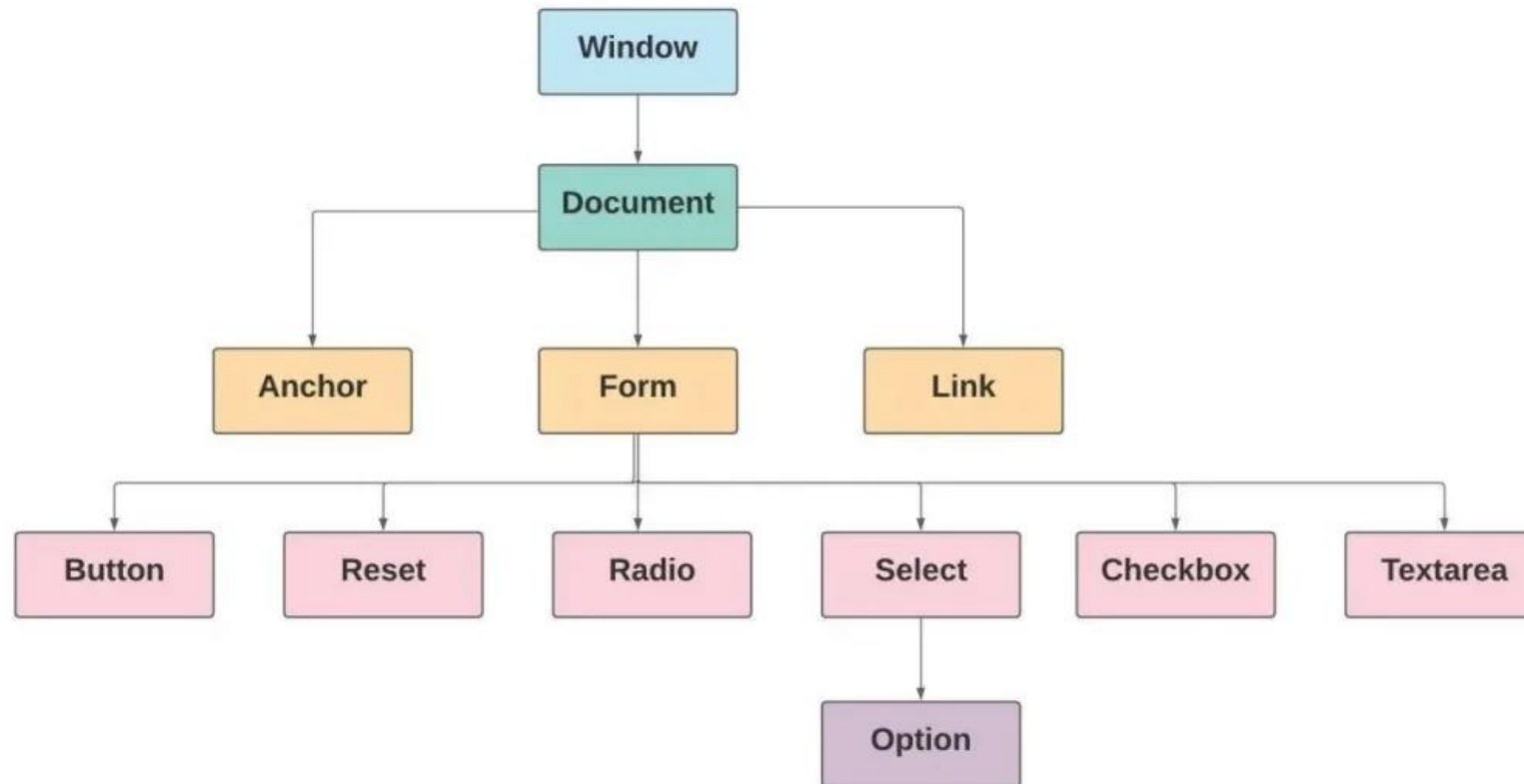
  <body>
    <h1>The heading</h1>
    <div>
      <p>A description</p>
      <p>A summary</p>
    </div>
  </body>
</html>
```

DOM

Document Object Model



Representation of HTML DOM



- **DOM (Document Object Model)** là một chuẩn do **W3C** đề xuất, được sử dụng để truy cập và thao tác với cấu trúc của tài liệu HTML và XML. Khi một trang web được tải, trình duyệt sẽ chuyển đổi toàn bộ tài liệu HTML thành mô hình đối tượng tài liệu (**DOM**), cho phép JavaScript hoặc các ngôn ngữ lập trình khác có thể truy cập, chỉnh sửa, và tương tác với nội dung của trang web một cách động.
- DOM là gì?
 - DOM là một mô hình dữ liệu dạng cây, trong đó các thành phần HTML được biểu diễn dưới dạng các đối tượng và nút (nodes).
 - Mỗi phần tử HTML, như thẻ <div>, <p>, , đều là một nút trong cây DOM. Các nút có thể chứa các thuộc tính, giá trị văn bản hoặc các phần tử con.

Ví dụ về cây DOM cho một tài liệu HTML đơn giản:

```
html
Sao chép mã

<!DOCTYPE html>
<html>
  <head>
    <title>DOM Example</title>
  </head>
  <body>
    <h1>Hello World</h1>
    <p>This is a simple paragraph.</p>
  </body>
</html>
```

Cấu trúc DOM tương ứng của ví dụ trên:

```
CSS
Sao chép mã

html
├── head
│   └── title
│       └── "DOM Example"
└── body
    ├── h1
    │   └── "Hello World"
    └── p
        └── "This is a simple paragraph."
        ↓
```


- Bạn có thể sử dụng JavaScript để truy cập và thao tác các phần tử trong cây DOM thông qua các phương thức như
 - getElementById()
 - getElementsByClassName()
 - querySelector()
 - querySelectorAll()

- JavaScript có thể sử dụng DOM để thay đổi nội dung, thuộc tính, và kiểu dáng của các phần tử trên trang web.
 - Thay đổi nội dung (Text Content)
 - Thay đổi thuộc tính (Attributes)
 - Thay đổi kiểu dáng (CSS)
 - Thêm, xóa và thay thế phần tử
 - Thêm phần tử mới (Create Elements)
 - Xóa phần tử (Remove Elements)
 - Thay thế phần tử (Replace Elements)

- Bạn có thể sử dụng DOM để bắt các sự kiện trên trang web, chẳng hạn như khi người dùng nhấp chuột, nhập văn bản, hoặc di chuyển chuột.
 - Sự kiện nhấp chuột (Click Event)
 - Sự kiện nhập liệu (Input Event)

- **Bài tập 1: Thay đổi nội dung của một phần tử**

- Tạo một trang HTML có một đoạn văn và một nút bấm. Khi nhấn vào nút, nội dung của đoạn văn sẽ được thay đổi.

- **Bài tập 2: Đếm số lần nhấp vào nút**

- Tạo một trang HTML với một nút và một đoạn văn. Mỗi khi bạn nhấn vào nút, số lần nhấn sẽ được hiển thị trong đoạn văn.

- **Bài tập 3: Thêm phần tử mới vào danh sách**

- Tạo một danh sách HTML (ul hoặc ol) và một ô nhập liệu. Khi người dùng nhập văn bản vào ô nhập liệu và nhấn vào nút, phần tử mới sẽ được thêm vào danh sách.

- **Bài tập 4: Thay đổi màu nền của trang**

- Tạo một trang HTML có 3 nút. Mỗi nút khi nhấn sẽ thay đổi màu nền của trang thành một màu khác nhau (ví dụ: đỏ, xanh, vàng).

- **Bài tập 5: Ẩn và hiện một phần tử**

- Tạo một đoạn văn và hai nút: "Hiện" và "Ẩn". Khi người dùng nhấn vào nút "Ẩn", đoạn văn sẽ biến mất. Khi nhấn vào nút "Hiện", đoạn văn sẽ xuất hiện trở lại.

- Tạo trang web có 1 button và 1 thẻ input type = color. Khi click button đó thì thay background-color của trang bằng màu hiện tại của input.

DOM Events

Mouse events

Keyboard events

TouchEvent

Event

- *Syntax*

element.addEventListener(event, function);

- *Example*

element.addEventListener("click", myFunction);

```
function myFunction() {  
    alert ("Hello World!");  
}
```

- Tạo một trang login, khi người dùng bấm phím Enter thì thực hiện chức năng login



The image shows a login form on a light orange background. It contains two text input fields, each preceded by the label 'User name'. The first input field is empty, and the second one contains the text 'User name'. Below the second input field is a button labeled 'Login'.

- parentNode
- childNodes[nodenumber]
- firstChild
- lastChild
- nextSibling
- previousSibling

- `document.createElement`
- `document.createTextNode`
- `element.appendChild(para);`
- `element.insertBefore(para, child);`
- `parent.removeChild(child);`
- `parent.replaceChild(para, child);`

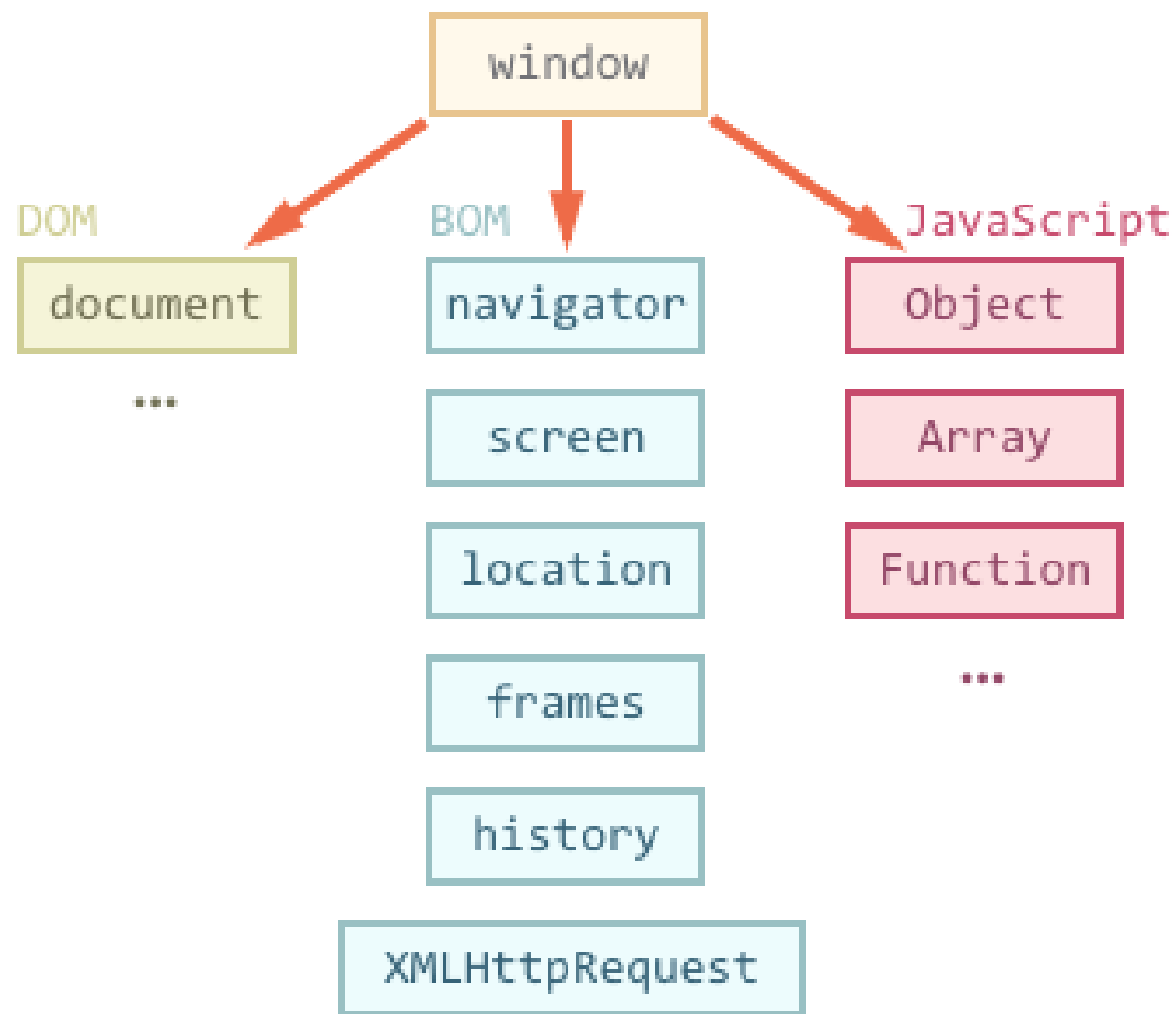
- Tạo trang quản lý danh sách công việc như hình vẽ cho phép nhập nội dung vào textbox và thêm nội dung vào list mỗi khi click button Add

My To Do List

Add

Hit the gym	×
✓ Pay bills	×
Meet George	×
Buy eggs	×
Read a book	×
Organize office	×

JS Browser BOM



- Định nghĩa 1 mảng Person (id, name, age, address, subjects, isActive). Subjects là 1 mảng các môn học mà person đó đã tham gia.
- Đọc dữ liệu mảng đã cho và hiển thị dạng bảng.

Xử lý bất đồng bộ (Asynchronous JavaScript)

Xử lý bất
đồng bộ

- setTimeout()
- setInterval()

Promises
và
async/await

- Promises
- async và await:
Giúp xử lý bất đồng
bộ dễ đọc hơn.

Xử lý lỗi và kiểm tra đầu vào

Xử lý lỗi với
try...catch

Kiểm tra dữ liệu:
Đảm bảo dữ liệu nhập
hợp lệ

Callback

- Callback Hell

Promise

- `then()`, `catch()`, `finally()`

Async – Await

- Cách viết bất đồng bộ hiện đại
- Xử lý lỗi bằng `try...catch`

Fetch API (thay thế AJAX)

- Lấy dữ liệu từ API thật (ví dụ: thời tiết)

- **localStorage / sessionStorage**
 - Lưu dữ liệu (giỏ hàng, ghi nhớ đăng nhập)
 - JSON stringify/parse

Hiệu ứng giao diện

- Toggle menu
- Dark mode
- Slider cơ bản

Form Validation nâng cao

- Regex
- Kiểm tra email, mật khẩu
- Hiển thị lỗi trực quan

- Arrow Function
- Template Literals
- Fetch API

- **Node.js (Express)**
 - Công ty sử dụng: **Netflix, Paypal, Uber, GoDaddy, Shopee**
 - **Ưu điểm:**
 - Siêu nhanh nhờ non-blocking I/O
 - Dễ học cho sinh viên đã biết JS
 - **Nhược điểm:**
 - Không phù hợp xử lý CPU nặng

- **NestJS**
 - Công ty sử dụng: **Adidas, Autodesk, Microsoft (một số team)**
 - **Ưu điểm:**
 - Kiến trúc module rõ ràng
 - Sử dụng TypeScript → chuẩn enterprise
 - **Nhược điểm:**
 - Học lâu hơn Express

- Axios – gọi API (meta, Shopee, Tiki)
- Lodash – xử lý dữ liệu nhanh & tiện
- Moment.js / Day.js – xử lý ngày tháng
- Chart.js / ECharts – vẽ biểu đồ như Grab, Shopee
- Socket.IO – xử lý realtime (chat, tracking shipper)

- **Xác định mục tiêu nghề nghiệp**
 - Frontend → học React/Vue
 - Backend → Node.js & Database
 - Fullstack → React + Node.js
- **Kiểm tra thị trường tuyển dụng**
 - Lên VietnamWorks, TopCV, ITviec → tìm "React" / "Vue" / "Angular"
- **Theo dõi xu hướng công nghệ**
 - Google Trends
 - StackOverflow Developer Survey
 - Github Stars
- **Tự làm mini project**
 - Website giới thiệu sản phẩm
 - Todo app
 - Giỏ hàng (cart)
 - Weather app (API)

- BẮT BUỘC
 - JavaScript nền tảng
 - DOM & Event
 - Fetch API + Async/Await
- NÊN HỌC SỚM
 - React
 - Node.js
 - Git & GitHub
 - TailwindCSS
- NÂNG CAO
 - TypeScript
 - Redux / Zustand
 - NestJS
 - CI/CD (GitHub Actions)

- Xây dựng app "To-Do List" gồm:
 - Thêm / xóa task
 - Lưu vào localStorage
 - Dùng module tách file (storage.js, ui.js)

- Tạo giao diện gồm:
 - 1 input chọn màu
 - 1 button “Apply Color”
 - Khi nhấn button → đổi màu nền trang thành màu đã chọn.

- Mục tiêu
 - Tạo object đại diện cho student
 - CRUD dữ liệu trên array
 - Tách code thành function
- Yêu cầu
 - Form nhập:
 - name
 - age
 - score
 - Chức năng:
 - Thêm sinh viên
 - Hiển thị danh sách dưới dạng table
 - Xóa sinh viên
 - Tính điểm trung bình lớp

- Mục tiêu
 - Thao tác danh sách dữ liệu
 - Render UI bằng JavaScript
 - Ứng dụng filter()
- Yêu cầu
 - Hiển thị danh sách sản phẩm
 - Có input “Giá tối đa”
 - Khi nhập giá $\rightarrow$ lọc sản phẩm giá $\leq$ input

- Mục tiêu
 - Fetch API
 - Async/await
 - Hiển thị dữ liệu từ server (OpenWeatherMap)
- Yêu cầu
 - Nhập tên thành phố
 - Gọi API → hiển thị:
 - Nhiệt độ
 - Độ ẩm
 - Mô tả thời tiết

- Mục tiêu
 - Render câu hỏi từ mảng object
 - Chấm điểm tự động
 - Chuyển câu hỏi, hiển thị kết quả cuối
- Yêu cầu chức năng
 - Next question
 - Kiểm tra đáp án
 - Tổng kết điểm



CMC UNIVERSITY

Aspire to Inspire the Digital World



THANK YOU